

Chatroom distribuée SOAP + REST

Architecture polyglotte avec frontend hexagonal

Mouhamed Lamine Faye Zeitune

École Supérieure Polytechnique — DGI

Cours Web Services — DIC3 — Mai 2026

Plan

1. Sujet et fonctionnalités
2. Conception et architecture
3. Modèle de données et intégrité
4. Communication SOAP
5. Communication REST
6. Bilan

Sujet du projet

Une chatroom textuelle multi-utilisateurs implémentée **deux fois** sur le même cas métier :

- **Phase 1** en SOAP (Java JAX-WS Metro + Node.js node-soap)
- **Phase 2** en REST (Java Jersey + Python Flask)

Un même frontend PHP consomme les deux backends grâce au pattern **Port et Adaptateurs**, sélectionné par variable d'environnement.

L'objectif est de comparer concrètement les deux paradigmes sur un cas identique.

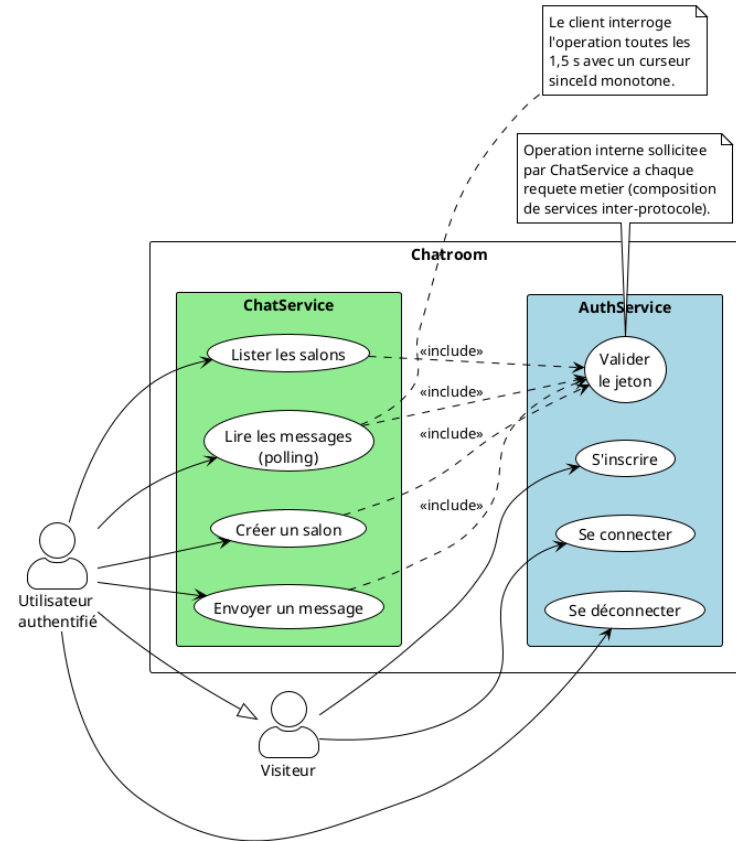
Fonctionnalités attendues

AuthService	ChatService
register (création de compte)	createRoom (création de salon)
login (jeton de session)	listRooms (lister les salons)
logout (invalidation)	sendMessage (publier)
validateToken (contrôle interne)	getMessages (polling)

Huit opérations métier au total, réparties en deux services autonomes.

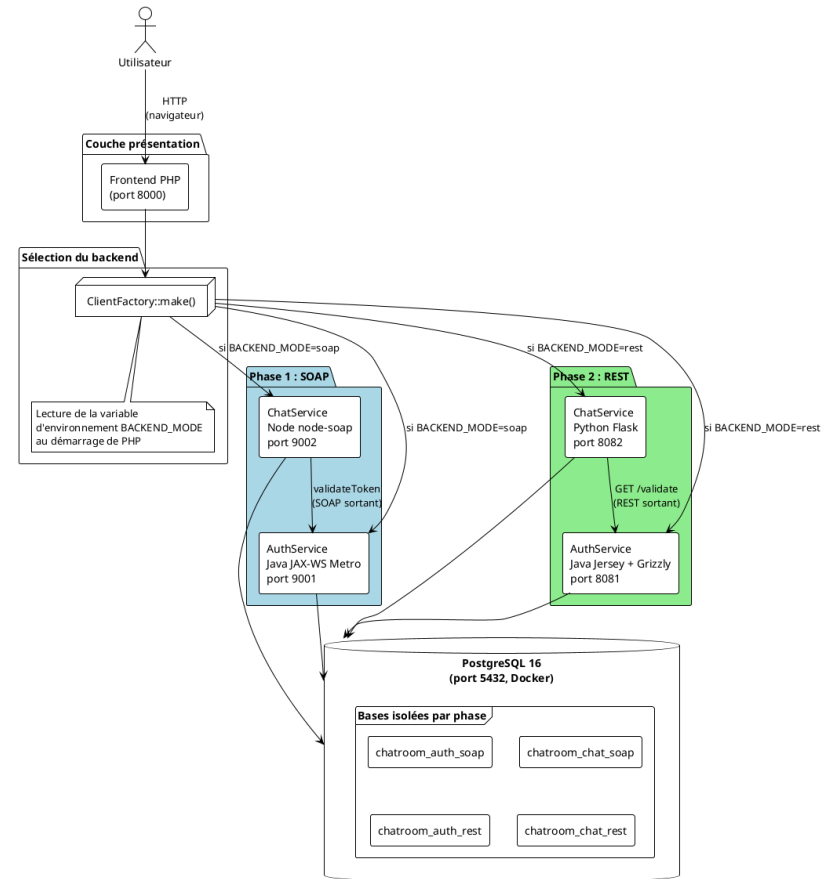
Cas d'utilisation

Diagramme de cas d'utilisation — Chatroom



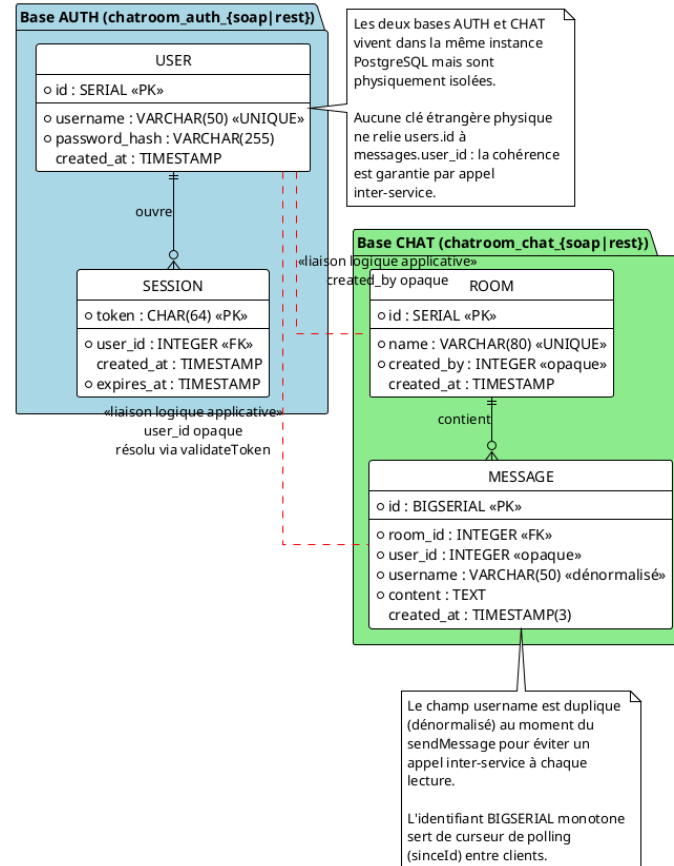
Architecture globale

Architecture globale du système



Modèle conceptuel de données

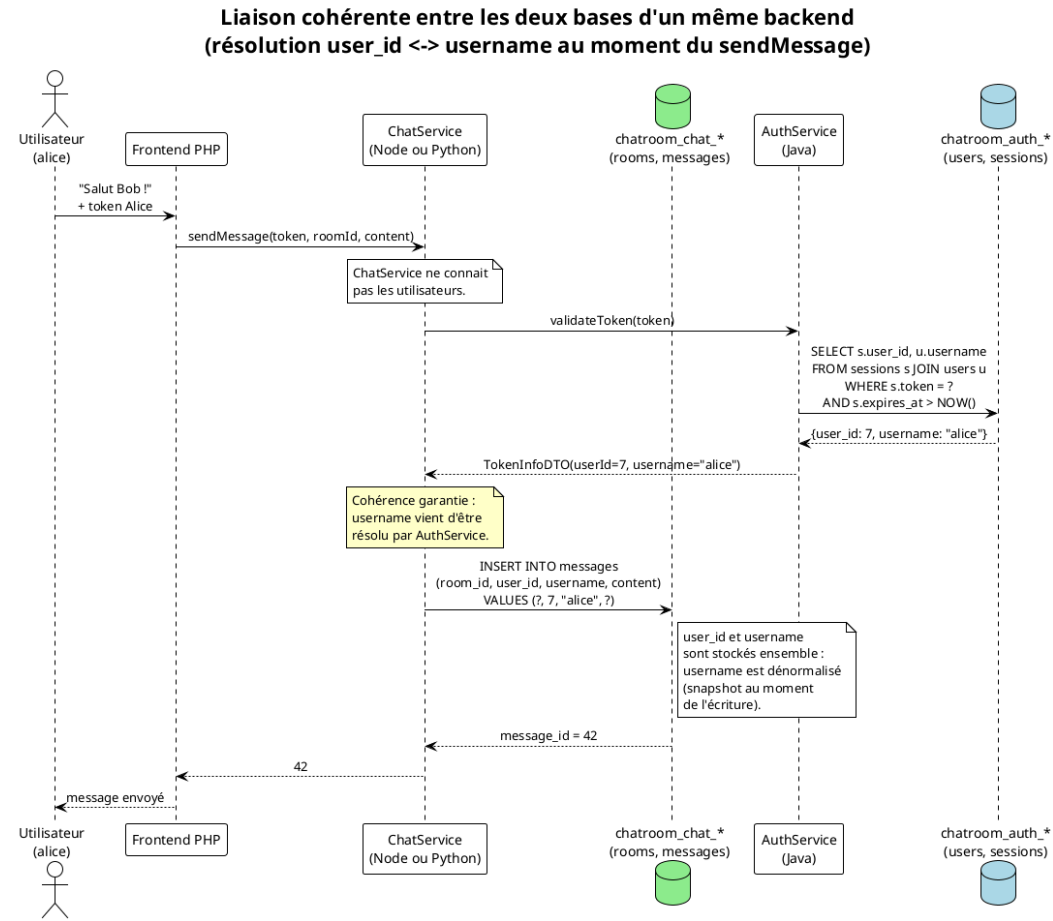
Modèle Conceptuel de Données (MCD) — backends isolés



Comment garantir l'intégrité sans clé étrangère ?

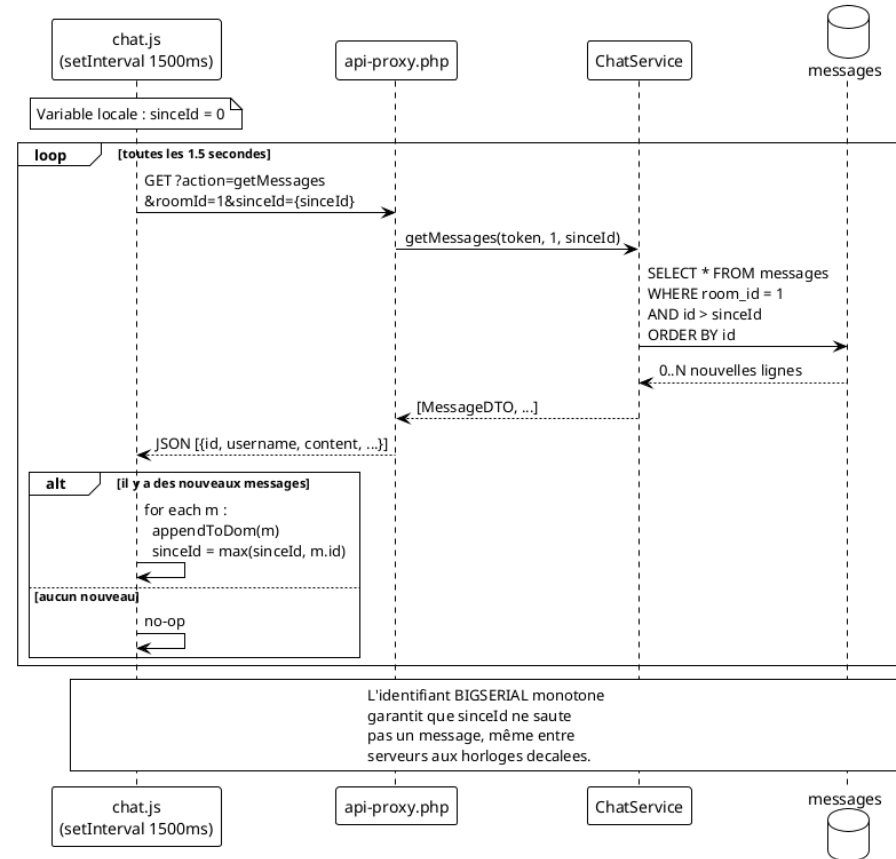
Les deux bases d'un même backend sont **physiquement isolées**.
PostgreSQL n'autorise pas de FK inter-base.

La solution : composition de services



Polling côté navigateur

Séquence du polling côté navigateur (toutes les 1.5 secondes)



Stack technique par bloc

Bloc	Phase SOAP	Phase REST
Frontend	PHP 8.2 (SoapClient)	PHP 8.2 (cURL)
AuthService	Java 21 + JAX-WS Metro	Java 21 + Jersey + Grizzly
ChatService	Node.js 20 + node-soap	Python 3.12 + Flask
BD	PostgreSQL 16 (2 bases)	PostgreSQL 16 (2 bases)
Sécurité	bcrypt (cost 10)	bcrypt (cost 10)

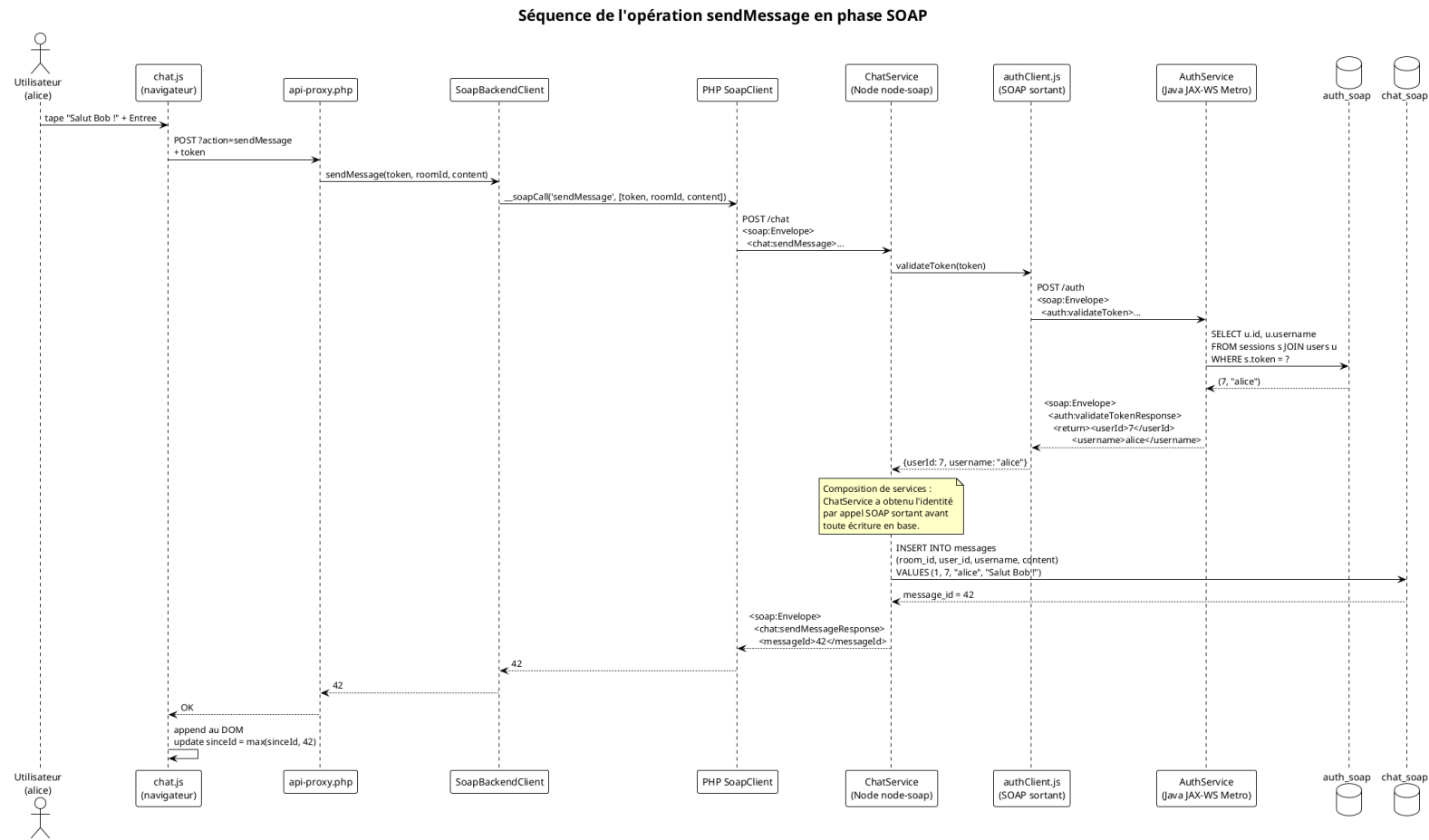
SOAP : le contrat WSDL

Un WSDL décrit publiquement un service SOAP en **cinq sections imbriquées** :

Section	Rôle
<code><types></code>	dictionnaire XSD des structures (DTOs)
<code><message></code>	regroupement logique des entrées et sorties
<code><portType></code>	interface abstraite : opérations + entrées + sorties + fautes
<code><binding></code>	protocole concret : SOAP sur HTTP, document/literal
<code><service></code>	adresse réelle de l'endpoint (URL)

Un client SOAP **génère ses stubs automatiquement** à partir du WSDL.

SOAP : envoi d'un message



SoapUI en action

Projet SoapUI : **14 TestCases**, 6 Property Transfers, 23 assertions.

Test du flux complet avec validation inter-service.

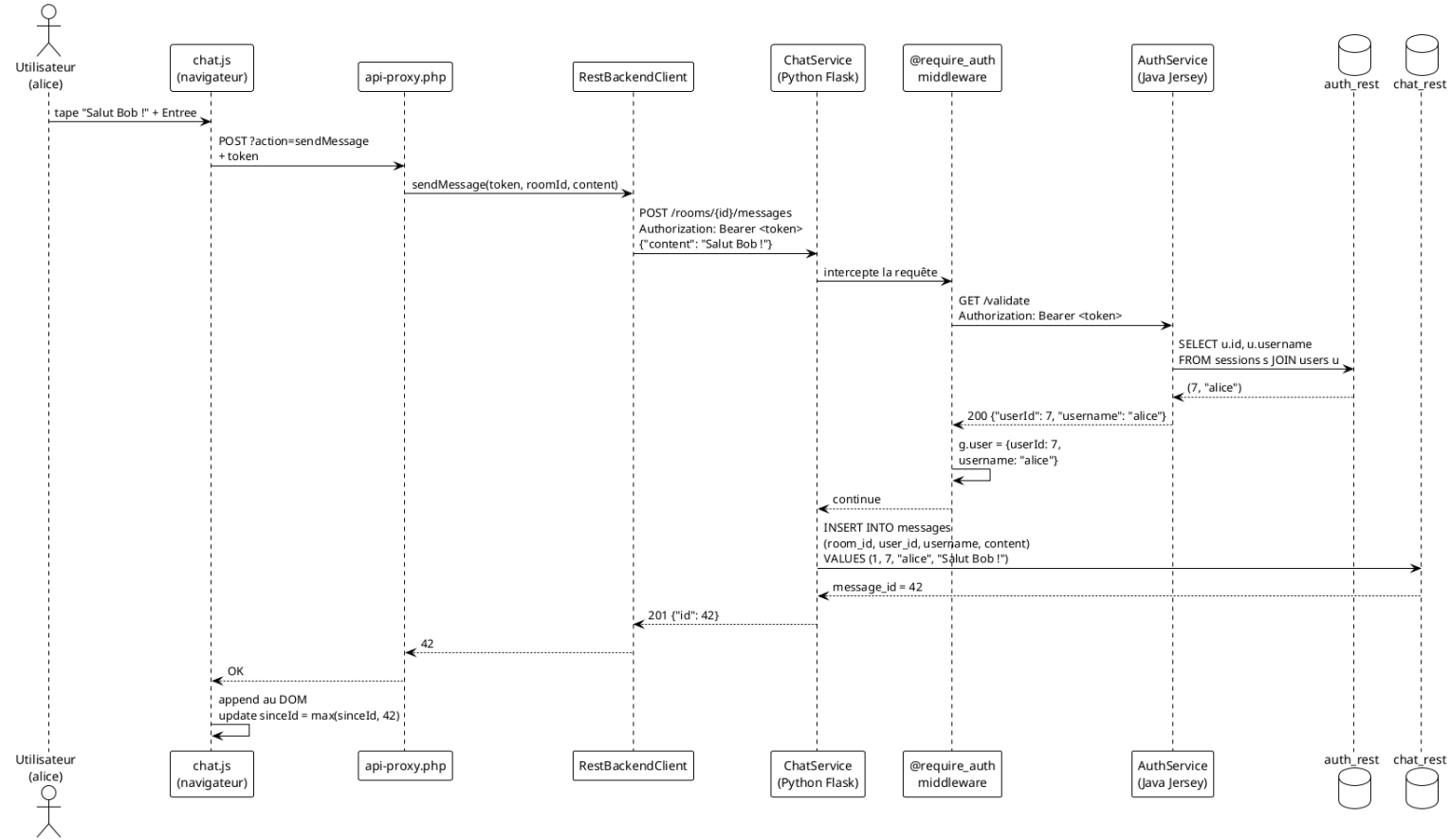
REST : principes

- **Ressources** identifiées par des URI (`/rooms` , `/rooms/{id}/messages`)
- **Verbes HTTP** : `POST` créer, `GET` lire, `DELETE` supprimer
- **Codes de statut** : 200, 201, 401, 404, 409
- **Authentication** : en-tête `Authorization: Bearer <token>`
- **Négociation de contenu** : un même endpoint sert JSON ou XML selon `Accept`

```
GET /rooms HTTP/1.1  
Authorization: Bearer abc...  
Accept: application/xml
```

REST : envoi d'un message

Séquence de l'opération sendMessage en phase REST



Postman en action

Collection Postman : **28 requêtes** organisées par service.

Chaque opération existe en **JSON et en XML**, exécutables en boucle via le Collection Runner.

Bilan

- **Composition de services** observable : chaque opération métier traverse les deux services et illustre la délégation de l'authentification.
- **Polyglossie tenue** : Java, Node.js et Python interagissent sans friction grâce aux contrats standardisés.
- **Bascule SOAP**

REST par simple variable d'environnement, sans modification du code applicatif, grâce au pattern **Port et Adaptateurs**.

Merci de votre attention.